Automated Quality Control with Transfer Learning – Manufacturing Defect Detection

This notebook demonstrates transfer learning using ResNet50 for binary defect detection in manufacturing.

```
import tensorflow as tf
from tensorflow.keras.preprocessing.image import ImageDataGenerator
from tensorflow.keras.applications import ResNet50
from tensorflow.keras.layers import GlobalAveragePooling2D, Dense, Dropout
from tensorflow.keras.models import Model
from tensorflow.keras.optimizers import Adam
import matplotlib.pyplot as plt
import numpy as np
from sklearn.metrics import classification_report
import os

print(tf.__version__)
```

```
2.20.0
```

Discovery Phase: Data Preparation

```
# Assume dataset is organized as:
# data/
#   train/
#     good/
#     defect/
#   test/
#     good/
#     defect/

# For demonstration (no real dataset provided), create a small synthetic dummy
# In production, download NEU Steel or MVTec AD and organize binary classes.

import os
import shutil
import numpy as np
from tensorflow.keras.preprocessing.image import array_to_img

def create_dummy_dataset(base_dir='data', num_samples=80):
    for split in ['train', 'test']:
        for cls in ['good', 'defect']:
```

```
            os.makedirs(os.path.join(base_dir, split, cls), exist_ok=True)

    # Generate dummy images
    for i in range(num_samples):
        # Good images: lower intensity
        good_img = np.random.rand(224, 224, 3) * 0.6 + 0.2
        array_to_img(good_img).save(os.path.join(base_dir, 'train', 'good', f'g
        # Defect images: higher contrast/artifact
        defect_img = np.random.rand(224, 224, 3)
        defect_img[80:150, 80:150] = 0.9  # simulate defect patch
        array_to_img(defect_img).save(os.path.join(base_dir, 'train', 'defect',

    # Simple test split (copy some)
    for i in range(20):
        shutil.copy(os.path.join(base_dir, 'train', 'good', f'good_{i}.png'),
                    os.path.join(base_dir, 'test', 'good', f'good_test_{i}.png'
        shutil.copy(os.path.join(base_dir, 'train', 'defect', f'defect_{i}.png'
                    os.path.join(base_dir, 'test', 'defect', f'defect_test_{i}.

create_dummy_dataset()
print('✅ Dummy dataset created successfully for demo.')

train_datagen = ImageDataGenerator(
    rescale=1./255,
    rotation_range=20,
    width_shift_range=0.2,
    height_shift_range=0.2,
    shear_range=0.2,
    zoom_range=0.2,
    horizontal_flip=True,
    validation_split=0.2
)

test_datagen = ImageDataGenerator(rescale=1./255)

train_generator = train_datagen.flow_from_directory(
    'data/train',
    target_size=(224, 224),
    batch_size=16,  # Smaller for dummy data
    class_mode='binary',
    subset='training'
)

validation_generator = train_datagen.flow_from_directory(
    'data/train',
    target_size=(224, 224),
    batch_size=16,
    class_mode='binary',
    subset='validation'
)
```

```
test_generator = test_datagen.flow_from_directory(
    'data/test',
    target_size=(224, 224),
    batch_size=16,
    class_mode='binary',
    shuffle=False
)
```

```
✅ Dummy dataset created successfully for demo.
Found 128 images belonging to 2 classes.
Found 32 images belonging to 2 classes.
Found 40 images belonging to 2 classes.
```

Technical Phase: Model Construction (Brain Swap)

```
# Load pre-trained ResNet50
base_model = ResNet50(weights='imagenet', include_top=False, input_shape=(224,

# Freeze the base model
base_model.trainable = False

# Add custom head
x = base_model.output
x = GlobalAveragePooling2D()(x)
x = Dense(128, activation='relu')(x)
x = Dropout(0.5)(x)
predictions = Dense(1, activation='sigmoid')(x)

model = Model(inputs=base_model.input, outputs=predictions)

model.compile(optimizer=Adam(learning_rate=0.001), loss='binary_crossentropy',
model.summary()
```

```
Downloading data from https://storage.googleapis.com/tensorflow/keras-applicat
94765736/94765736 ━━━━━━━━━━━━━━━━━━━━ 1s 0us/step
Model: "functional"
```

Layer (type)	Output Shape	Param #	Connected to
input_layer (InputLayer)	(None, 224, 224, 3)	0	-
conv1_pad (ZeroPadding2D)	(None, 230, 230, 3)	0	input_layer[0][0]
conv1_conv (Conv2D)	(None, 112, 112, 64)	9,472	conv1_pad[0][0]
conv1_bn (BatchNormalizatio…	(None, 112, 112, 64)	256	conv1_conv[0][0]
conv1_relu (Activation)	(None, 112, 112, 64)	0	conv1_bn[0][0]
pool1_pad (ZeroPadding2D)	(None, 114, 114, 64)	0	conv1_relu[0][0]
pool1_pool (MaxPooling2D)	(None, 56, 56, 64)	0	pool1_pad[0][0]
conv2_block1_1_conv (Conv2D)	(None, 56, 56, 64)	4,160	pool1_pool[0][0]
conv2_block1_1_bn (BatchNormalizatio…	(None, 56, 56, 64)	256	conv2_block1_1_c…
conv2_block1_1_relu (Activation)	(None, 56, 56, 64)	0	conv2_block1_1_b…
conv2_block1_2_conv (Conv2D)	(None, 56, 56, 64)	36,928	conv2_block1_1_r…
conv2_block1_2_bn (BatchNormalizatio…	(None, 56, 56, 64)	256	conv2_block1_2_c…
conv2_block1_2_relu (Activation)	(None, 56, 56, 64)	0	conv2_block1_2_b…
conv2_block1_0_conv (Conv2D)	(None, 56, 56, 256)	16,640	pool1_pool[0][0]
conv2_block1_3_conv (Conv2D)	(None, 56, 56, 256)	16,640	conv2_block1_2_r…
conv2_block1_0_bn (BatchNormalizatio…	(None, 56, 56, 256)	1,024	conv2_block1_0_c…
conv2_block1_3_bn (BatchNormalizatio…	(None, 56, 56, 256)	1,024	conv2_block1_3_c…

conv2_block1_add (Add)	(None, 56, 56, 256)	0	conv2_block1_0_b… conv2_block1_3_b…
conv2_block1_out (Activation)	(None, 56, 56, 256)	0	conv2_block1_add…
conv2_block2_1_conv (Conv2D)	(None, 56, 56, 64)	16,448	conv2_block1_out…
conv2_block2_1_bn (BatchNormalizatio…	(None, 56, 56, 64)	256	conv2_block2_1_c…
conv2_block2_1_relu (Activation)	(None, 56, 56, 64)	0	conv2_block2_1_b…
conv2_block2_2_conv (Conv2D)	(None, 56, 56, 64)	36,928	conv2_block2_1_r…
conv2_block2_2_bn (BatchNormalizatio…	(None, 56, 56, 64)	256	conv2_block2_2_c…
conv2_block2_2_relu (Activation)	(None, 56, 56, 64)	0	conv2_block2_2_b…
conv2_block2_3_conv (Conv2D)	(None, 56, 56, 256)	16,640	conv2_block2_2_r…
conv2_block2_3_bn (BatchNormalizatio…	(None, 56, 56, 256)	1,024	conv2_block2_3_c…
conv2_block2_add (Add)	(None, 56, 56, 256)	0	conv2_block1_out… conv2_block2_3_b…
conv2_block2_out (Activation)	(None, 56, 56, 256)	0	conv2_block2_add…
conv2_block3_1_conv (Conv2D)	(None, 56, 56, 64)	16,448	conv2_block2_out…
conv2_block3_1_bn (BatchNormalizatio…	(None, 56, 56, 64)	256	conv2_block3_1_c…
conv2_block3_1_relu (Activation)	(None, 56, 56, 64)	0	conv2_block3_1_b…
conv2_block3_2_conv (Conv2D)	(None, 56, 56, 64)	36,928	conv2_block3_1_r…
conv2_block3_2_bn (BatchNormalizatio…	(None, 56, 56, 64)	256	conv2_block3_2_c…
conv2_block3_2_relu (Activation)	(None, 56, 56, 64)	0	conv2_block3_2_b…
conv2_block3_3_conv (Conv2D)	(None, 56, 56, 256)	16,640	conv2_block3_2_r…
conv2_block3_3_bn	(None, 56, 56,	1,024	conv2_block3_3_c…

(BatchNormalizatio…	256)		
conv2_block3_add (Add)	(None, 56, 56, 256)	0	conv2_block2_out… conv2_block3_3_b…
conv2_block3_out (Activation)	(None, 56, 56, 256)	0	conv2_block3_add…
conv3_block1_1_conv (Conv2D)	(None, 28, 28, 128)	32,896	conv2_block3_out…
conv3_block1_1_bn (BatchNormalizatio…	(None, 28, 28, 128)	512	conv3_block1_1_c…
conv3_block1_1_relu (Activation)	(None, 28, 28, 128)	0	conv3_block1_1_b…
conv3_block1_2_conv (Conv2D)	(None, 28, 28, 128)	147,584	conv3_block1_1_r…
conv3_block1_2_bn (BatchNormalizatio…	(None, 28, 28, 128)	512	conv3_block1_2_c…
conv3_block1_2_relu (Activation)	(None, 28, 28, 128)	0	conv3_block1_2_b…
conv3_block1_0_conv (Conv2D)	(None, 28, 28, 512)	131,584	conv2_block3_out…
conv3_block1_3_conv (Conv2D)	(None, 28, 28, 512)	66,048	conv3_block1_2_r…
conv3_block1_0_bn (BatchNormalizatio…	(None, 28, 28, 512)	2,048	conv3_block1_0_c…
conv3_block1_3_bn (BatchNormalizatio…	(None, 28, 28, 512)	2,048	conv3_block1_3_c…
conv3_block1_add (Add)	(None, 28, 28, 512)	0	conv3_block1_0_b… conv3_block1_3_b…
conv3_block1_out (Activation)	(None, 28, 28, 512)	0	conv3_block1_add…
conv3_block2_1_conv (Conv2D)	(None, 28, 28, 128)	65,664	conv3_block1_out…
conv3_block2_1_bn (BatchNormalizatio…	(None, 28, 28, 128)	512	conv3_block2_1_c…
conv3_block2_1_relu (Activation)	(None, 28, 28, 128)	0	conv3_block2_1_b…
conv3_block2_2_conv (Conv2D)	(None, 28, 28, 128)	147,584	conv3_block2_1_r…
conv3_block2_2_bn (BatchNormalizatio…	(None, 28, 28, 128)	512	conv3_block2_2_c…